Operating System Structure

Week - 02

Operating Systems

Instructor: Noreen Khalid

Chapter Outline

- Process Management
- Memory Management
- File Management
- I/O System Management
- Secondary Storage Management
- Networking
- System Protection
- Operating System Services
- OS Layered Approach
  - OS/2 Layer Structure
- Virtual Machines
- System Design Goals
- Mechanisms and Policies
- Operating System Implementation
- System Generation (SYSGEN)

OS Service Management

Process Management

- A process is a program in execution
- A process needs certain resources, including CPU time, memory, files, and I/O devices, to accomplish its task
- The operating system is responsible for the following activities in connection with process manager
  - Process creation and termination
  - Process suspension and resumption
  - Provision of mechanisms for:
    - process synchronization
    - process communication

Memory Management

- Memory is a large array of words or bytes, each with its own address
- It is a repository of quickly accessible data shared by the CPU and I/O devices.
- Main memory is a volatile storage device
  - It looses its contents in the case of system failure
- The operating system is responsible for the following activities in connections with memory management:
  - Keep track of which parts of memory are currently being used and by whom
  - Decide which processes to load when memory space becomes available
  - Allocate and deallocate memory space as needed

File Management

- A file is a collection of related information defined by its creator
- Commonly, files represent programs (both source and object forms) and data
- The operating system is responsible for the following activities in connections with file management:
  - File creation and deletion
  - Directory creation and deletion
  - Support of primitives for manipulating files and directories
  - Mapping files onto secondary storage
  - File backup on stable (nonvolatile) storage media

I/O System Management

- Operating system is responsible for I/O system management
- The I/O system consists of:
  - A buffer-caching system
  - A general device-driver interface
  - Drivers for specific hardware devices

•**Buffer-Caching System** → Smooth, efficient data transfer.
•**General Device-Driver Interface** → Provides uniform I/O API for applications.
•**Device Drivers** → Translate OS requests into hardware-specific actions.

Secondary Storage Management

- Since main memory (primary storage) is volatile and too small to accommodate all data and programs permanently, the computer system must provide secondary storage to back up main memory
- Most modern computer systems use disks as the principle on-line storage medium, for both programs and data
- The operating system is responsible for the following activities in connection with disk management:
  - Free space management
  - Storage allocation
  - Disk scheduling

Secondary Storage Management by OS includes:

1. **Free space management** → Track unused blocks.
2. **Storage allocation** → Decide how to store files.
3. **Disk scheduling** → Optimize I/O request order.
4. **File system management** → Provide structure for data.
5. **Reliability & recovery** → Ensure data safety.

Networked System

- A networking system is a collection of processors
- The processors in the system are connected through a communication network
- Communication takes place using a protocol
- This system provides user access to various system resources
- Access to a shared resource allows:
  - Computation speed-up
  - Increased data availability
  - Enhanced reliability

A **networked system** = interconnected computers managed by OS to provide:

- **Communication** (sockets, RPCs)
- **Resource sharing** (files, printers, applications)
- **Security** (authentication, encryption)
- **Fault tolerance** (distributed reliability)

Real-Life Examples of Networked Systems

1.University or Office LAN (Local Area Network)

1. All computers in labs or offices are connected.
2. Shared **printers, files, and applications** across the network.
3. Example: Student submits assignment from one PC → teacher accesses it on another.

- **Small-scale examples** → University LAN, Office network.
- **Large-scale examples** → Cloud storage, Banking networks, E-commerce, Social media.

System Protection

- Protection refers to a mechanism for controlling access by programs, processes, or users to both system and user resources
- The protection mechanism must:
  - distinguish between authorized and unauthorized access
  - specify the controls to be imposed
  - provide a means of enforcement

Example 1: University LAN (Local Area Network)

•In a university, multiple computers in labs are **connected via LAN**.

•The **networked OS** allows:

- **Communication** → Students can send messages or submit assignments through a central server.
- **Resource Sharing** → Printers, projectors, and lab software are shared across all PCs.
- **Security** → Only students with valid logins can access lab systems.
- **Fault Tolerance** → If one lab PC crashes, the rest of the network still works.

•**LAN in a university** → small-scale networked system.

•**Google Drive/Cloud** → large-scale, distributed networked system.

Operating System Services

Operating System Services

An **Operating System (OS)** provides a set of **services** to users, programs, and hardware to make the system convenient, efficient, and secure.
These services act as an **interface** between the user/programs and the computer hardware.

Major Operating System Services

1.Program Execution

1. Load programs into memory and run them.
2. Handle execution, termination, and errors.
   Example: Running MS Word or a Python script.

2.I/O Operations

1. Provide simple and uniform access to I/O devices.
   Example: Reading from keyboard, writing to a file, printing a document.

3.File System Manipulation

1. Create, delete, read, write, and manage files/directories.
   Example: Saving a Word document on disk.

Operating System Services

4.Communication

1. Allow processes to exchange information.
2. Methods: **Shared memory, Message passing, Sockets**.
   Example: Chat applications (WhatsApp, Messenger).

5.Error Detection

1. Detect and handle errors in hardware (disk failure, bad memory) and software (invalid memory access).
   Example: Blue screen error in Windows, “Segmentation fault” in Linux.

6.Resource Allocation

1. Manage CPU, memory, and I/O devices among multiple processes.
   Example: OS decides which process gets CPU time next.

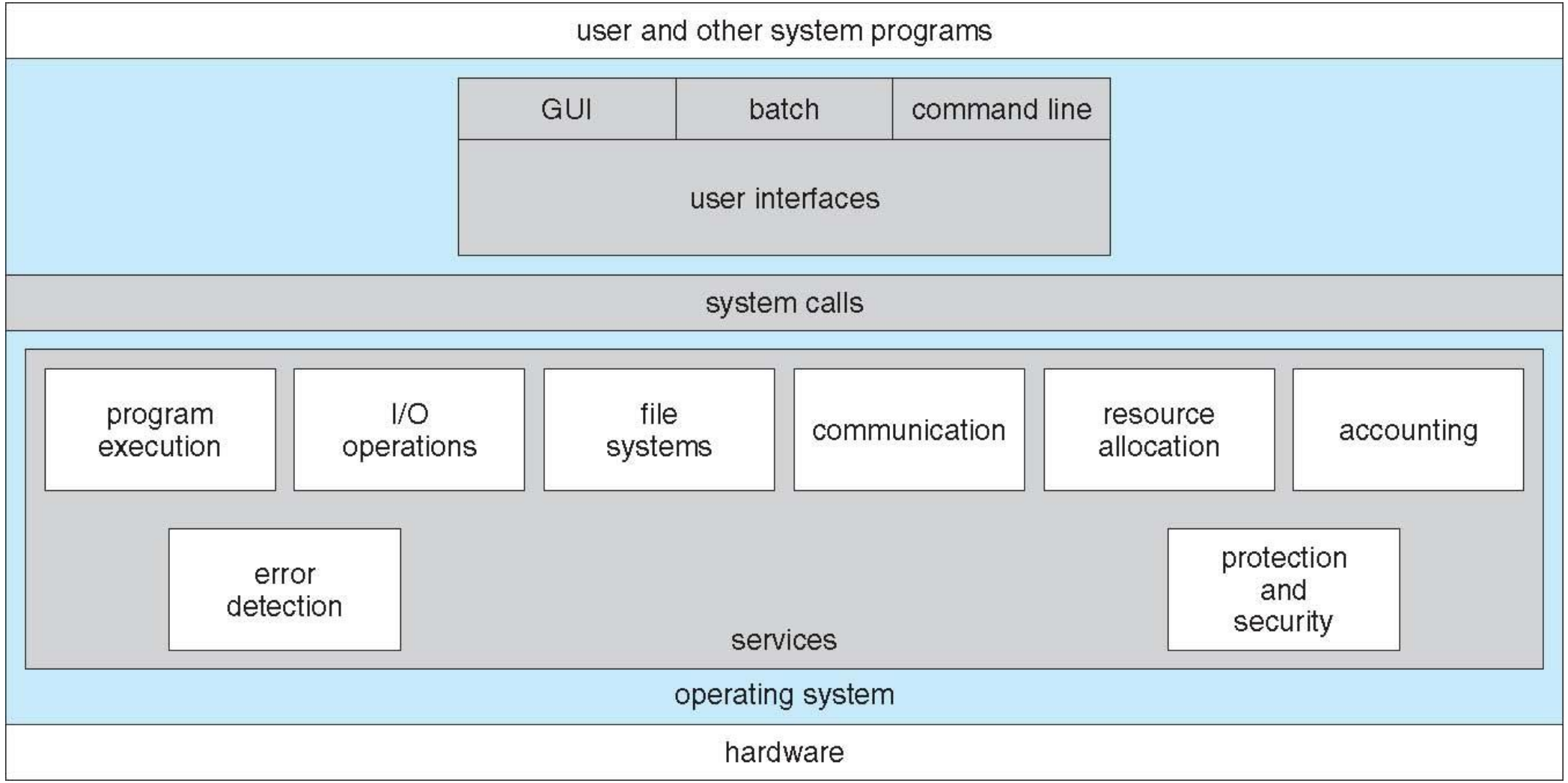
7.Accounting (Monitoring & Logging)

1. Keep track of resource usage by users/processes.
   Example: How much CPU time a process used, internet data usage.

8.Protection and Security

1. Control access to resources.
2. Ensure only authorized users/programs can access files, memory, or devices.
   Example: Login authentication, file permissions in Linux .

A View of Operating System Services



System Programs

- System programs are **utility programs** that provide a convenient environment for program development and execution.
  They sit **above the OS kernel** and **below the application software**.
- They can be divided into:
  - File manipulation
  - Status information sometimes stored in a File modification
  - Programming language support
  - Program loading and execution
  - Communications
  - Background services
- Most users' view of the operation system is defined by system programs, not the actual system calls

They are helper programs that make the OS more useful for both users and developers.

System Programs

Types of System Programs

1.File Management Programs
- •Create, delete, read, write, copy, and move files.
  Example: Windows Explorer, Linux cp, mv, rm.

2.Status Information Programs
- •Provide system info, performance, and resource usage.
  Example: Task Manager in Windows, top or uptime in Linux.

3.File Modification Programs
- •Edit or modify file content.
  Example: Notepad, nano, vim.

4.Programming Language Support
- •Compilers, assemblers, interpreters.
  Example: GCC (C compiler), Python interpreter.

5.Program Loading and Execution
- •Load programs into memory and run them.
  Example: ./a.out in Linux, double-clicking .exe in Windows.

System Programs

6. **Communication Programs**
   - •Enable communication between processes or users.
     Example: Email client, chat applications, ssh.
7. **Application Support**
   - •Provide common functions for apps.
     Example: Database engines, window systems.

Examples of System Programs (Real Life)

- •**Windows** → Notepad, Task Manager, Explorer, CMD.
- •**Linux/Unix** → vi, gcc, bash, ls, top.

System programs = "Helper utilities" provided by the OS for file handling, program development, system status, and user interaction.
They make the computer more **user-friendly** and support both **users & developers**.

Operating System Design and Implementation

Operating System Design and Implementation

- Design and Implementation of OS not “solvable”, but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- Start by defining goals and specifications
- Affected by choice of hardware, type of system
- User goals and System goals

OS Design Goals

User goals

Operating system should be

- convenient to use
- easy to learn
- reliable
- safe
- fast

System goals

Operating system should be

- easy to design
- implement
- maintain
- flexible
- reliable
- error-free
- efficient

Operating System Design and Implementation

- Important principle to separate
  - **Policy**: ***What*** will be done?
  - **Mechanism**: ***How*** to do it?
- Mechanisms determine how to do something, policies decide what will be done
  - The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later
- Specifying and designing OS is highly creative task of **software engineering**

Operating System Implementation

- Much variation
  - Early OSes in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- Actually usually a mix of languages
  - Lowest levels in assembly
  - Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- Code written in a high-level language:
  - can be written faster
  - is more compact
  - is easier to understand and debug
- An operating system is far easier to port (move to some other hardware) if it is written in a high-level language

Operating System Debugging

- **Debugging** is finding and fixing errors, or **bugs**
- OS generates **log files** containing error information
- Core Dump
  - Failure of an application can generate **core dump** file capturing memory of the process
- Crash Dump
  - Operating system failure can generate **crash dump** file containing kernel memory
- Beyond crashes, performance tuning can optimize system performance
  - Sometimes using ***trace listings*** of activities, recorded for analysis
  - **Profiling** is periodic sampling of instruction pointer to look for statistical trends

Operating System Generation (SYSGEN)

- Operating systems are designed to run on any of a class of machines
  - the system must be configured for each specific computer site
- **SYSGEN** program obtains information concerning the specific configuration of the hardware system
  - Used to build system-specific compiled kernel or system-tuned
  - Can generate more efficient code than one general kernel

System Boot

What is Booting?

- **Booting** = The process of **starting a computer and loading the operating system** into memory (RAM).
- It happens every time you **power on** or **restart** your computer.

Types of Booting

- **Cold Boot** → Starting the computer after power is completely off.
- **Warm Boot** → Restarting the computer without turning off power (e.g., pressing *Restart*).

Real-Life Examples

- **Windows PC** → Press power button → BIOS runs → Bootloader loads Windows kernel → Login screen.
- **Linux** → BIOS → GRUB → Linux kernel → shell/desktop environment.
- **Smartphones** → Boot ROM → bootloader → Android/iOS kernel → home screen.

GRUB is the Linux bootloader that lets you pick an OS, loads the kernel, and helps in recovery/debugging.

System Boot = The **step-by-step process of turning on a computer, running BIOS/UEFI, loading the bootloader, kernel, and finally the operating system.**

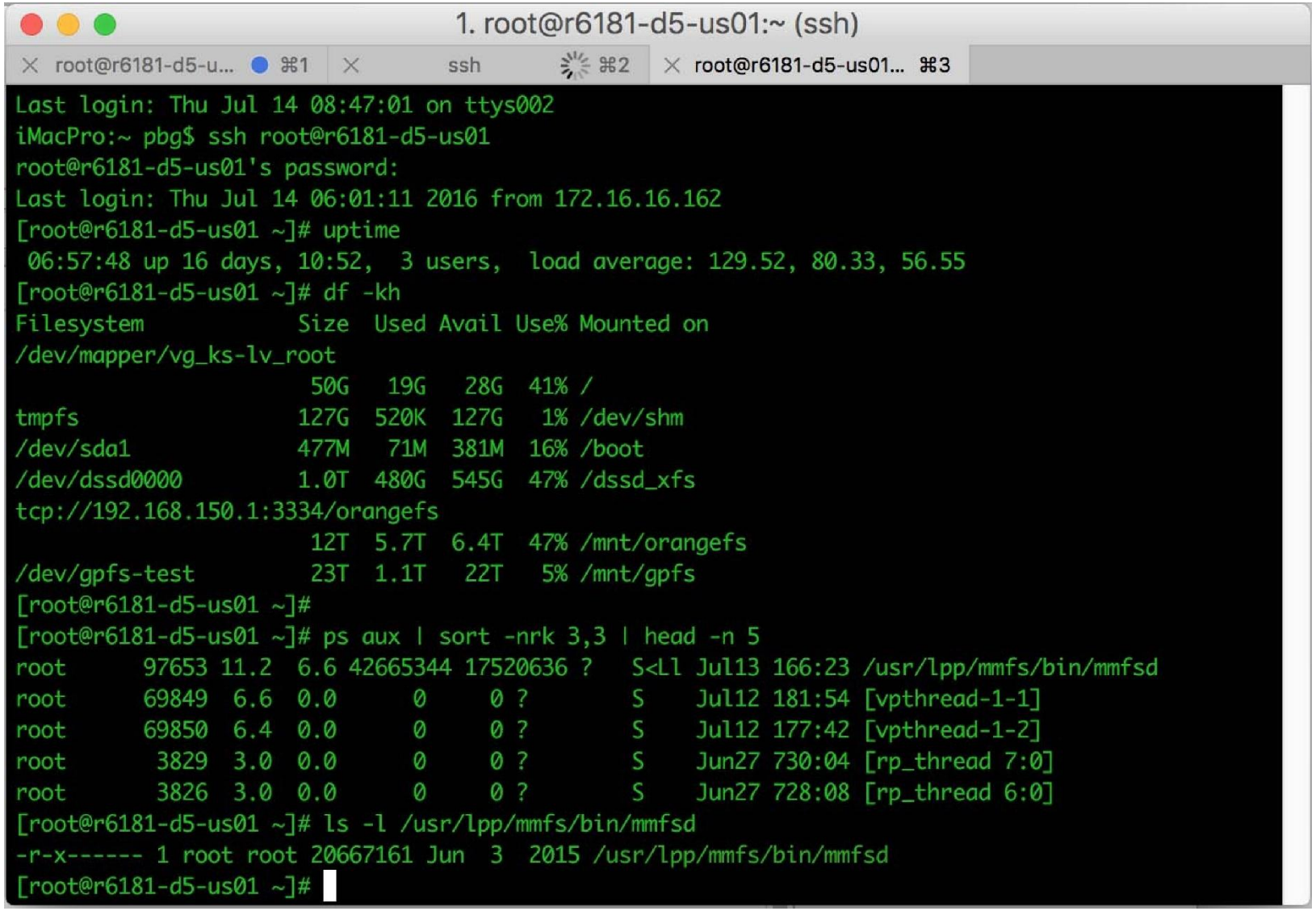
OS User Interface

Compare GUI, Batch, and CLI

Feature	GUI (Graphical)	CLI (Command Line)	Batch Processing
User Interaction	Through icons, menus, windows	Through text commands	No direct interaction (jobs submitted)
Ease of Use	Very easy, beginner-friendly	Harder, requires learning commands	Easy for repetitive jobs, but not interactive
Speed	Slower than CLI	Faster for experts	Efficient for large repetitive tasks
Resource Usage	High (needs CPU & memory for GUI)	Low	Medium
Examples	Windows, macOS, Android	Linux Terminal, MS-DOS, PowerShell	Cron jobs, Windows batch files (.bat)

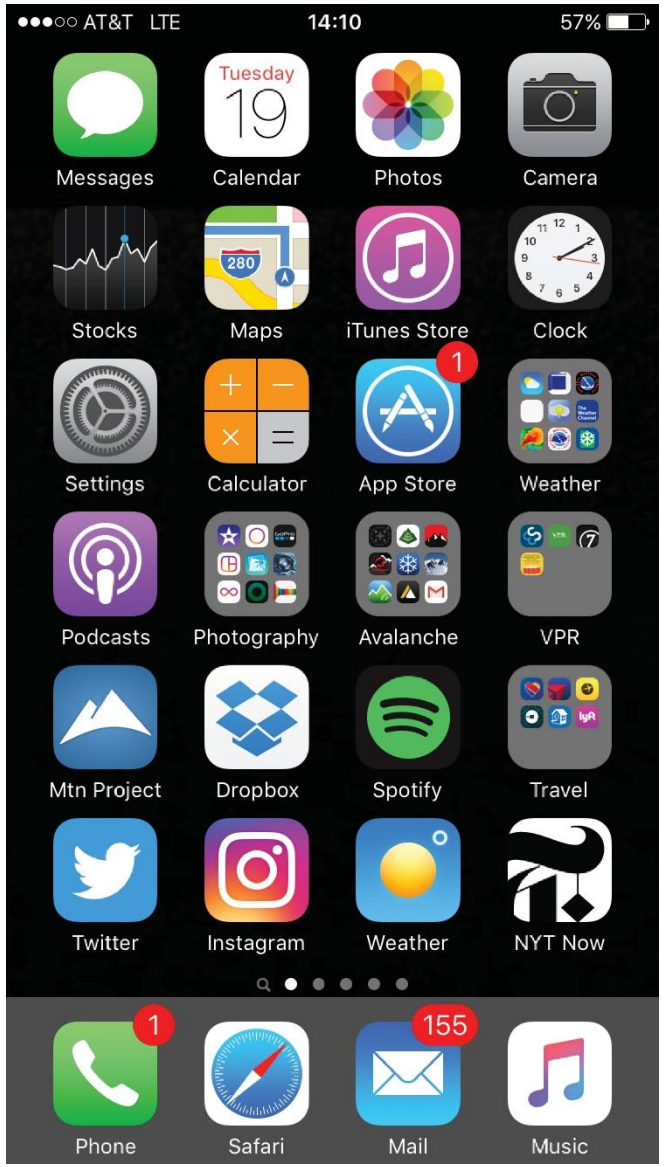
GUI = Easy, visual (ATM, smartphones, desktops).
CLI = Text-based, powerful (CMD (Type dir in Command Prompt), Linux terminal, networking).
Batch = Automatic, non-interactive (payroll, banking, scheduled backups).

Bourne Shell Command Interpreter



Touch-screen Interface

- Touch-screen devices require new interfaces
  - Mouse not possible or not desired
  - Actions and selection based on gestures
  - Virtual keyboard for text entry
- Many systems now include both CLI and GUI interfaces
  - Microsoft Windows is GUI with CLI "command" shell
  - Apple Mac OS X is "Aqua" GUI interface with UNIX kernel underneath and shells available
  - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)



Operating System Structure

Operating System Structure

- General-purpose OS is very large program
- Various ways to structure one as follows

Simple Structure

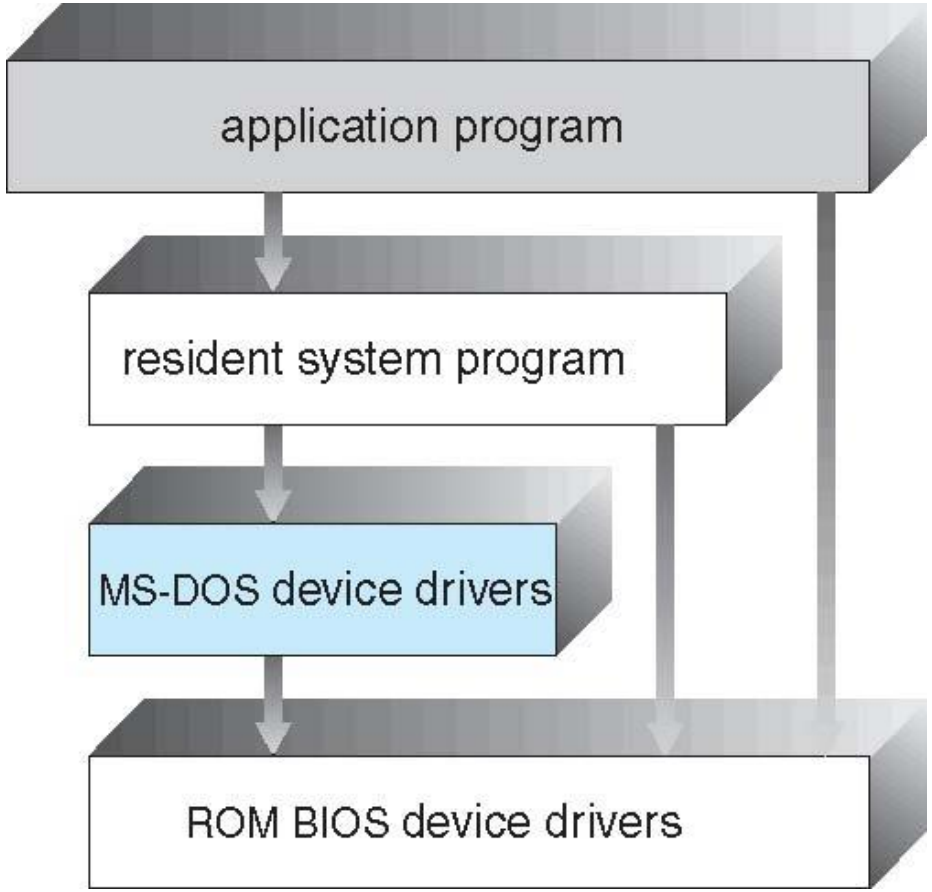
- I.e. MS-DOS – **MS-DOS (Microsoft Disk Operating System)** is an example of a **simple (monolithic) operating system structure**.
- It was widely used in the **1980s and early 1990s** before Windows became popular.
- **1. Simple Structure**
- **Definition**: No proper modular separation; all parts are mixed.
- **Real Examples**:
  - **MS-DOS** → All functions like file management, device drivers, system calls bundled together.
  - **Early UNIX** (Version 6).
- **Daily Life Link**: Like a **small calculator** program → everything in one place.

Real-Life Example / Where Used

•**ATM Machines**: Early ATM systems ran lightweight versions of DOS.

•**Old PCs**: Before Windows 95, personal computers booted directly into MS-DOS.

•**Embedded Systems**: Some older industrial control systems and cash registers used MS-DOS due to its simplicity and low resource requirements.



•No proper modular separation → everything is mixed.

•Example: **MS-DOS**, early UNIX.

•**Pros**: Easy to design.

•**Cons**: Hard to maintain, less secure.

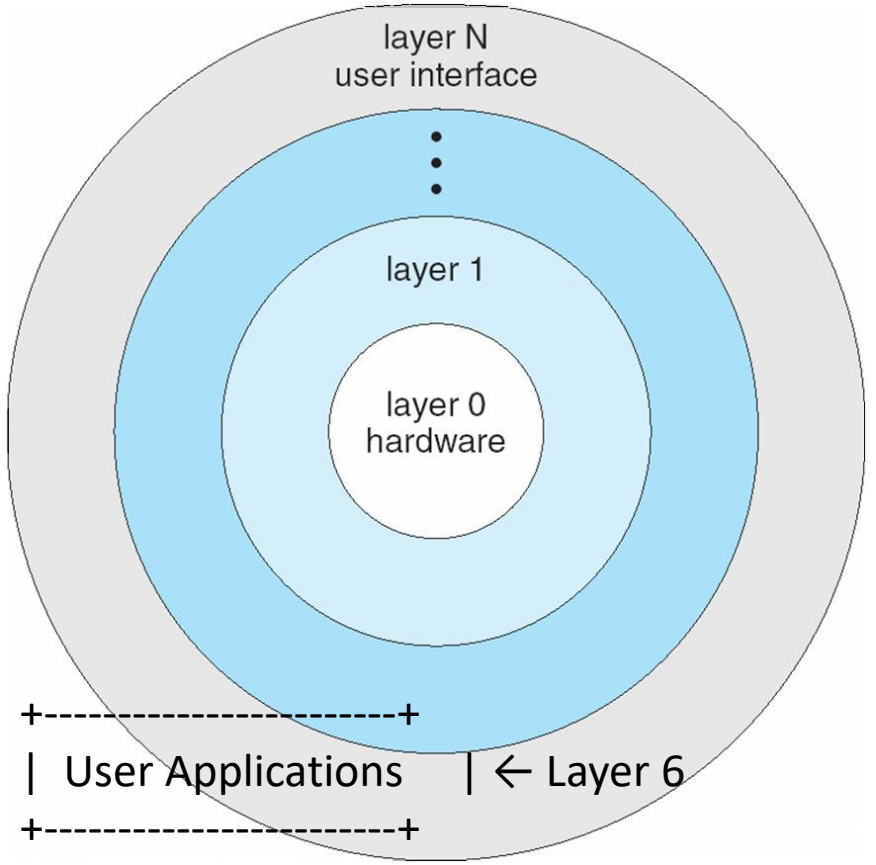
OS Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers
  - The bottom-most layer (layer 0), is the hardware
  - The highest (layer N) is the user interface
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

•OS is divided into **layers**, each built on top of the lower one.
•Each layer uses services of the layer below it and provides services to the layer above it.

Example: **THE OS, early UNIX variants**.
•**Pros**: Modularity, easier debugging.
•**Cons**: Overhead from too many layers.

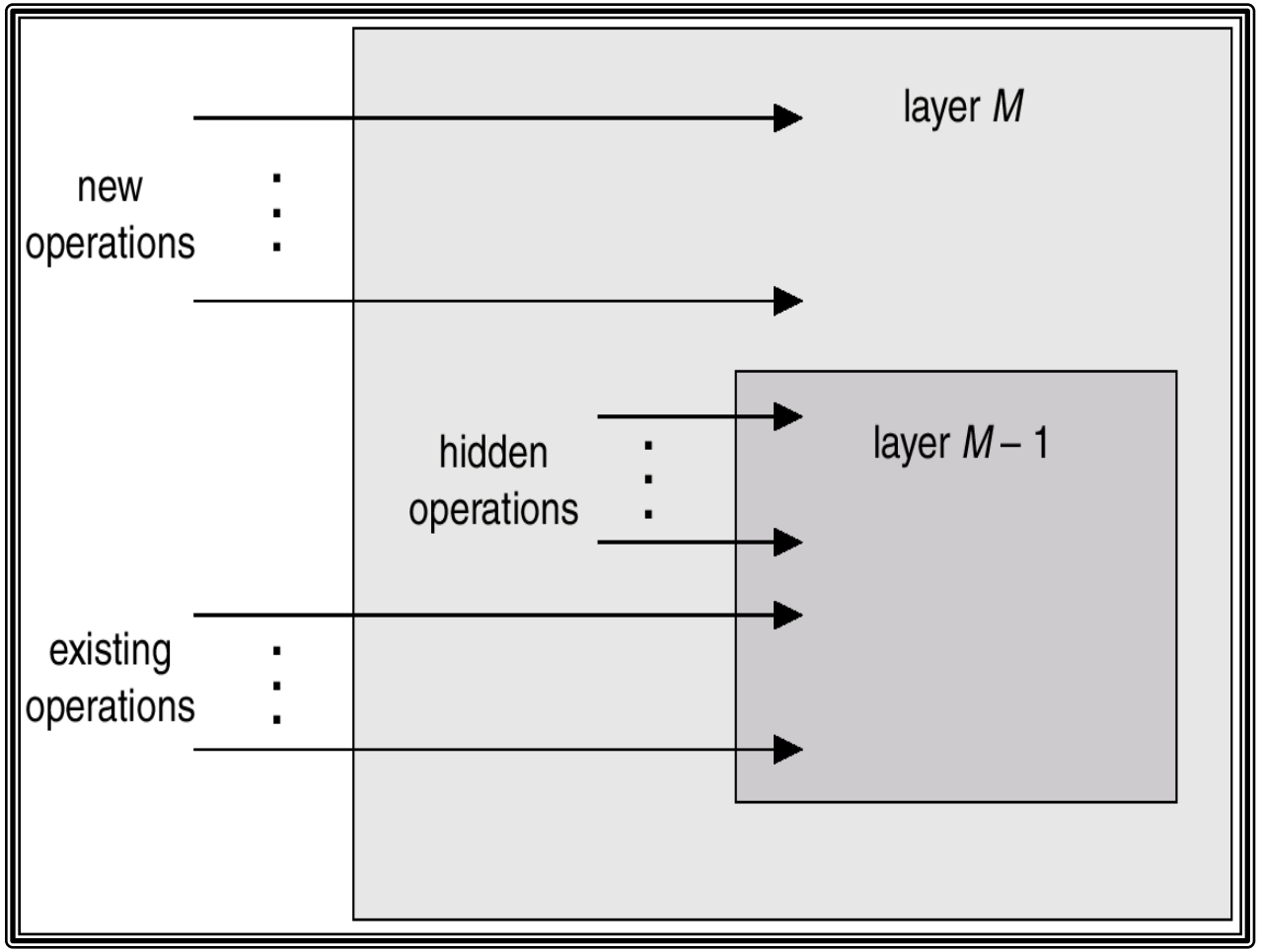
•**Daily Life Link**: Like a **cake with layers** → each layer builds on the previous one.



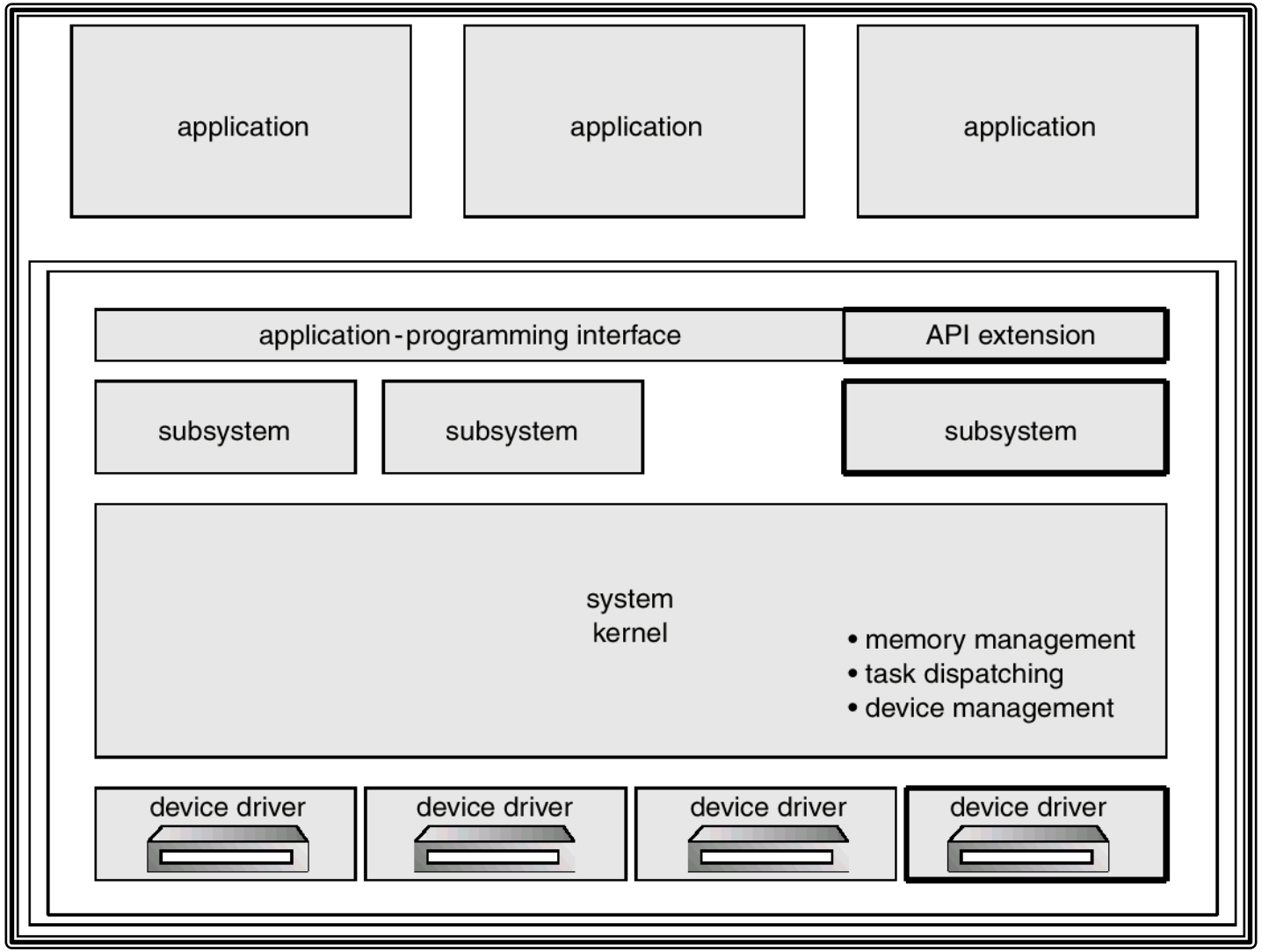

```
+----------------------+
|  User Applications   | ← Layer 6
+----------------------+
|  System Programs     | ← Layer 5
+----------------------+
|  I/O Management      | ← Layer 4
+----------------------+
|  Memory Management   | ← Layer 3
+----------------------+
|  Process Management  | ← Layer 2
+----------------------+
|  Hardware            | ← Layer 1
+----------------------+
```

An Operating System Layer

Direction	Meaning
Upward (Layer M → Higher Layers)	Provides *new* operations or services
Downward (Layer M → Layer M–1)	Uses *existing* operations provided by the lower layer
Hidden Operations	Internal helper routines invisible to higher layers



OS/2 Layer Structure

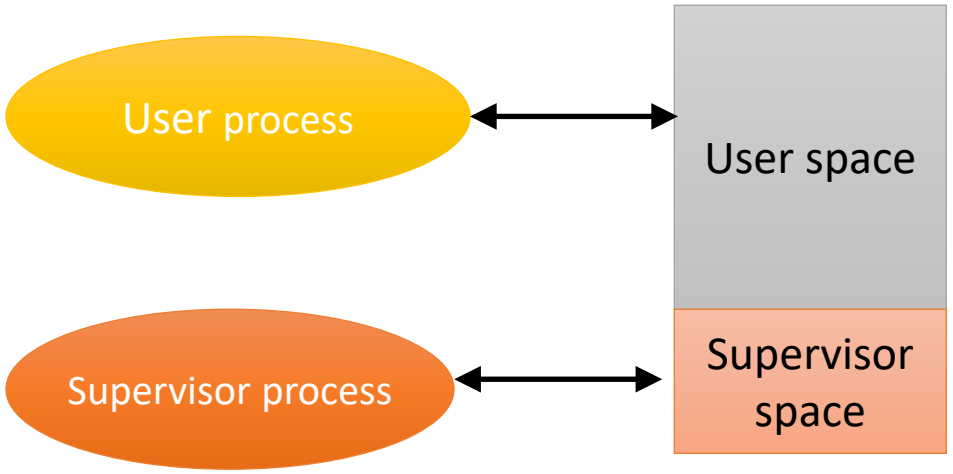


Kernel & Kernel Mode

- Kernel is part of operating system that includes most heavily used functions
- Generally, kernel is permanently in main memory
- Kernel Mode
  - It is a privileged, nucleus or supervisory mode into which the machine switches when the OS is running
  - Enables the execution of such privileged instructions
    - Such as in the area of I/O operations, which are not available to user mode
- The techniques by which a user program requests the kernel's services are;
  - System calls
  - Message passing
- The process can only be switched into kernel mode by the above actions

Kernel & Kernel Mode

- No other way that a user program can execute kernel instructions
- This facility is not given to user program in order to enforce system security



Supervisor & user memory

Concept	Meaning
Kernel	Core part of OS managing all hardware and system tasks
Kernel Mode	Privileged execution mode for kernel operations
User Mode	Restricted mode for running user programs
Mode Switch	Happens when a system call is made (User → Kernel → User)

Monolithic Structure – Original UNIX

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring
- The UNIX OS consists of two separable parts
  - Systems programs
  - The kernel
    - Consists of everything below the system-call interface and above the physical hardware
    - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

Definition: Whole OS runs in kernel mode as one big program.

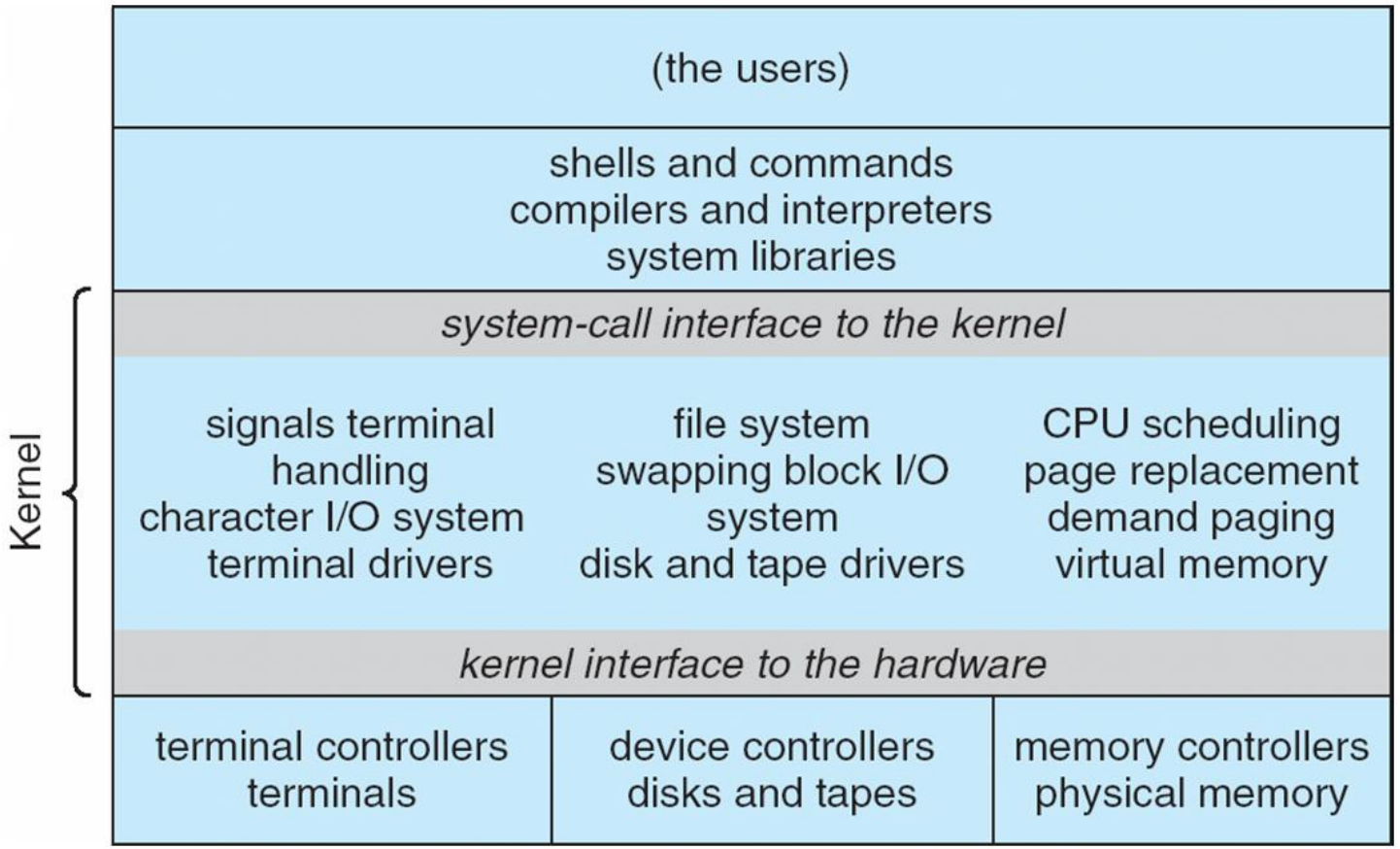
•**Real Examples**:

- **Linux (Ubuntu, Fedora, etc.)**
- **UNIX**
- **MS-DOS**

•**Daily Life Link**: Like a **single big factory** where all workers (modules) share the same space → fast but risky if one fails.

Traditional UNIX System Structure

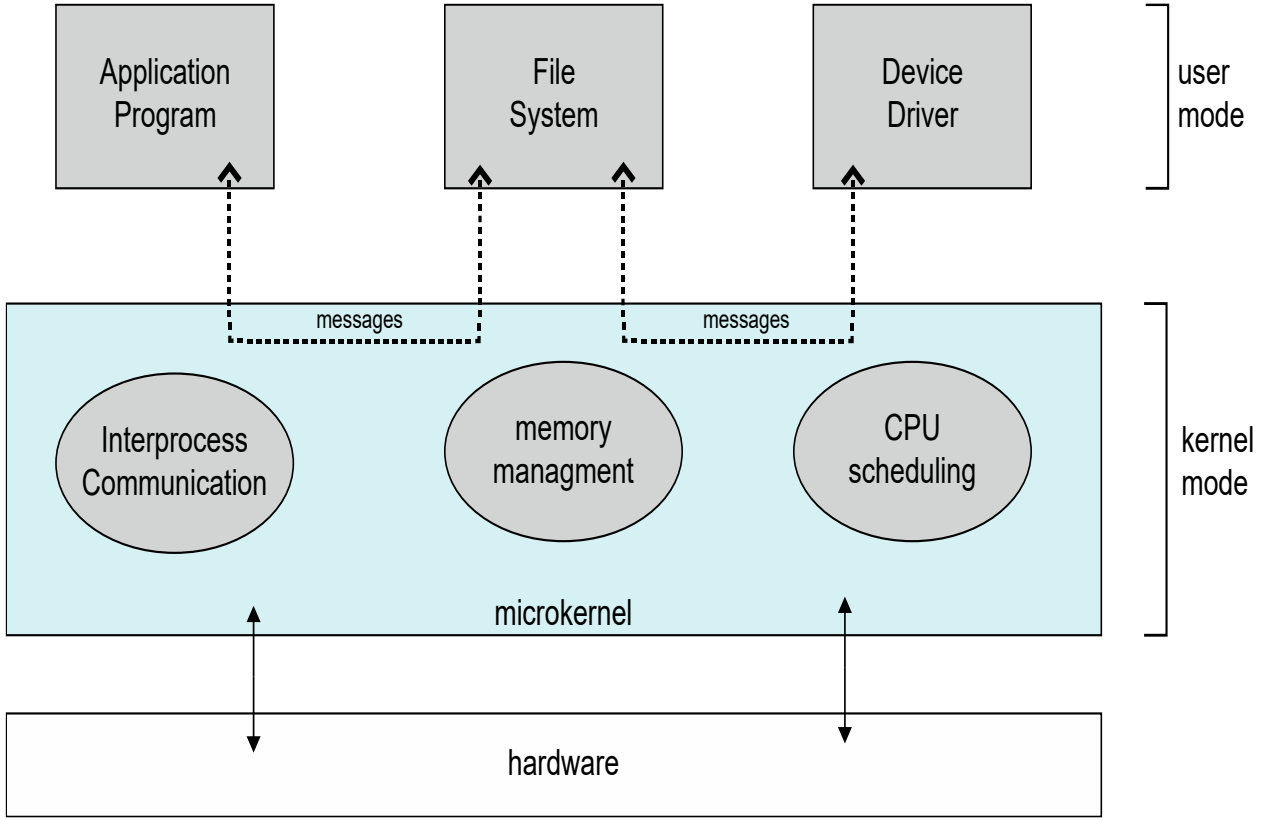
Beyond simple but not fully layered



Microkernel System Structure

- Only **essential services** (CPU scheduling, memory, inter-process communication) run in kernel.
- Other services (drivers, file systems) run in **user space**.
- Example: **Minix, QNX, macOS (partially)**.
- **Pros**: More secure, modular, reliable.
- **Cons**: Slower (extra communication between kernel and user space).

- **Real Examples**:
- **Minix** (educational OS).
- **QNX** (used in cars, medical devices).
- **macOS/iOS (XNU kernel is hybrid but microkernel-based)**.
- **Daily Life Link**: Like a **company HQ** with only managers (kernel) inside → other departments (drivers, file system) in separate offices (user space).

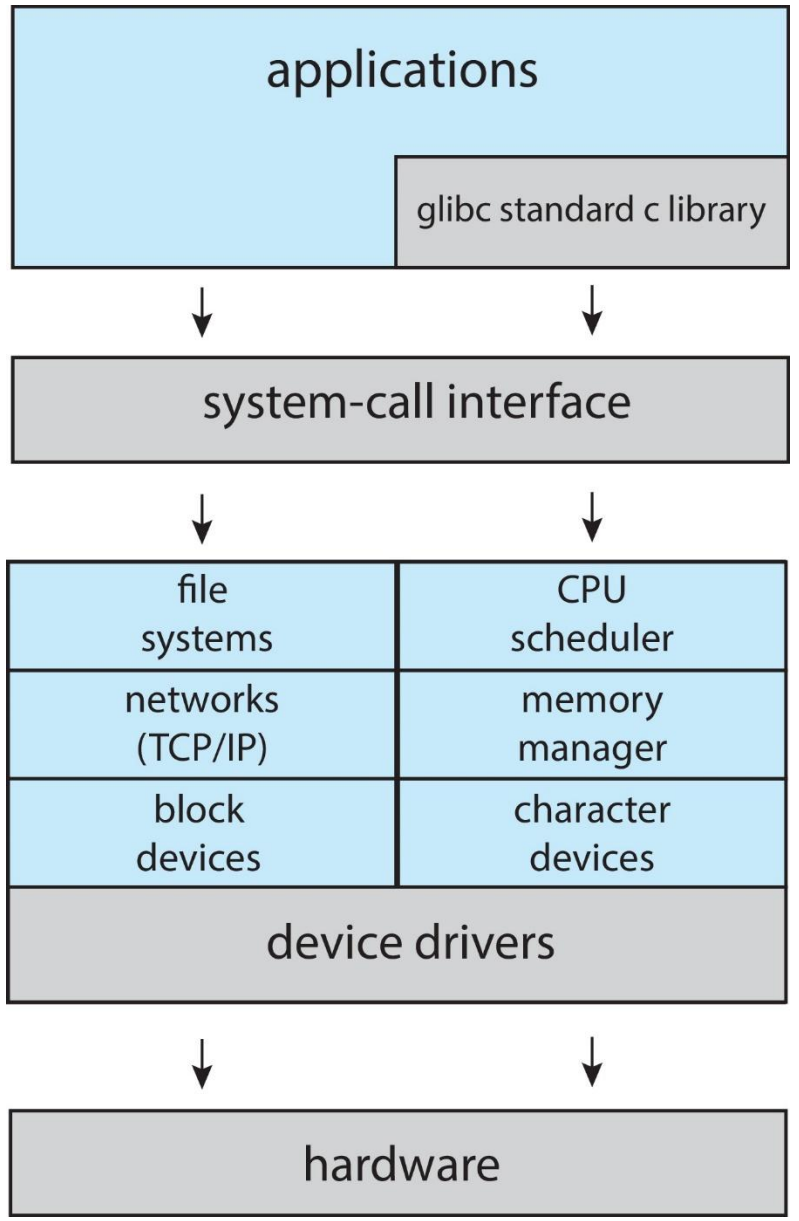


Modules

- A **module** is a **separate, independent part** of an operating system that performs a **specific function**.
  The **modular approach** means the OS is **divided into multiple components (modules)**, each responsible for a particular task like process management, memory management, I/O, etc.
- Each module **can be loaded, updated, or replaced independently** without affecting the rest of the system.
- Most modern operating systems implement loadable kernel modules
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
  - Linux, Solaris, etc

Linux System Structure

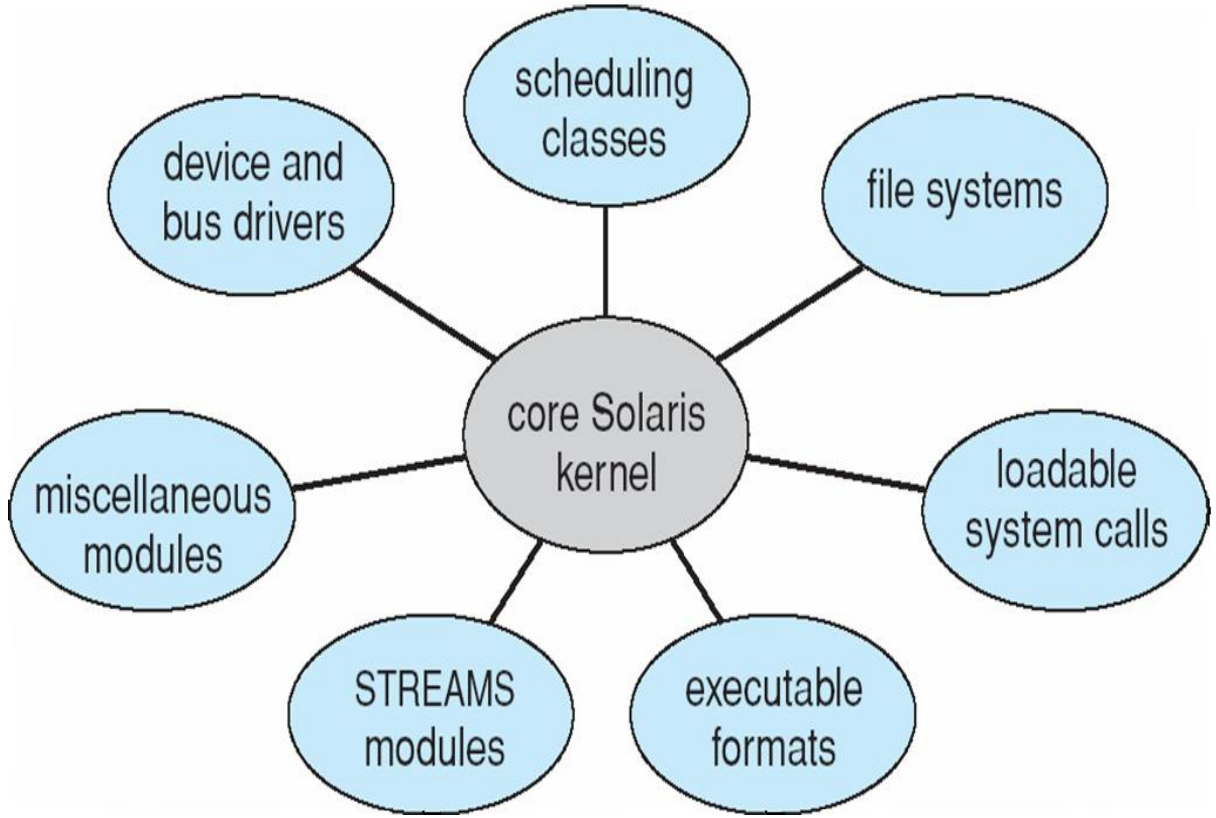
- Monolithic plus modular design



Solaris Modular Approach

Solaris is a **UNIX-based operating system** originally developed by **Sun Microsystems** (now owned by Oracle).
It uses a **modular architecture** meaning the system is divided into **independent modules**, each performing a specific function.
This makes Solaris **flexible**, **scalable**, and **easy to maintain**.

Solaris's modular structure is designed for **large-scale, high-performance, and continuously running systems**.
It's widely used in **servers, data centers, banking systems, and telecom environments** where reliability and flexibility are critical.



Hybrid Systems

A **Hybrid System** (or **Hybrid Kernel**) is a type of OS architecture that **combines the best features of both**:

- **Monolithic systems** (high performance, direct hardware access), and
- **Microkernel systems** (modularity, stability, and security).

In short:

Hybrid = Monolithic + Microkernel + Modular design elements

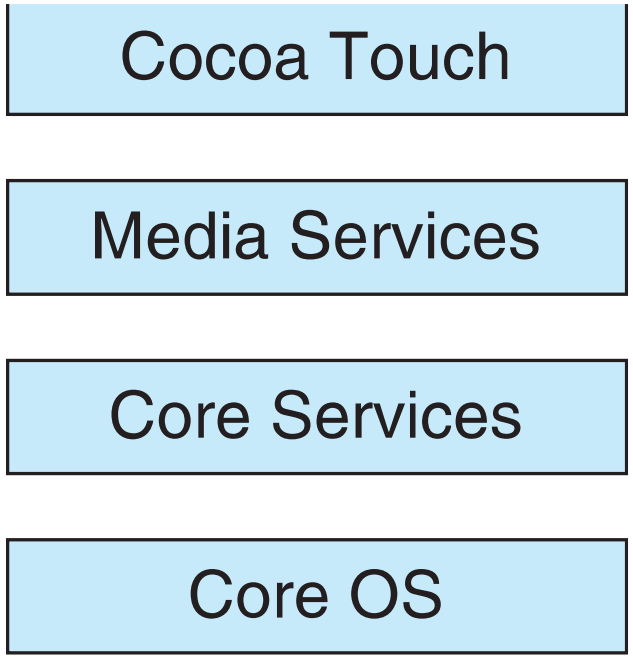
Example 1 – **Windows 10 / 11**
Real-life usage: Almost every **personal computer, laptop, or enterprise server** today.

Example 2 – **macOS (XNU Kernel)**
Real-life usage: All **Apple devices** (MacBook, iMac) use this hybrid approach.

Example 3 – **iOS**
Used in **iPhones and iPads** — combines real-time responsiveness with system stability.

iOS

- **iOS** is Apple's **mobile operating system** used in **iPhones, iPads, and iPod Touch**.
  It is derived from **macOS (Mac OS X)** but **optimized for touch interfaces, mobile hardware, and ARM processors**.



Real-Life Example

When you open **Apple Music** on an iPhone:

1. The **Cocoa Touch**(UIKit) layer handles touch and UI.
2. The **Media layer** plays audio and displays album art.
3. The **Core Services** layer connects to iCloud and Apple Music servers.
4. The **Core OS** layer manages memory, battery, and audio hardware

Android

- **Android** is an **open-source mobile operating system** developed by **Google** for smartphones, tablets, TVs, and wearables.
  It's based on a **modified Linux kernel** and designed mainly for **touchscreen devices**.

Layer	Description	Example
Applications	User-facing apps	WhatsApp, Instagram
Application Framework	API support for apps	Notification Manager
Android Runtime & Libraries	App execution and core libraries	SQLite, WebKit
Linux Kernel	Hardware management	Camera drivers

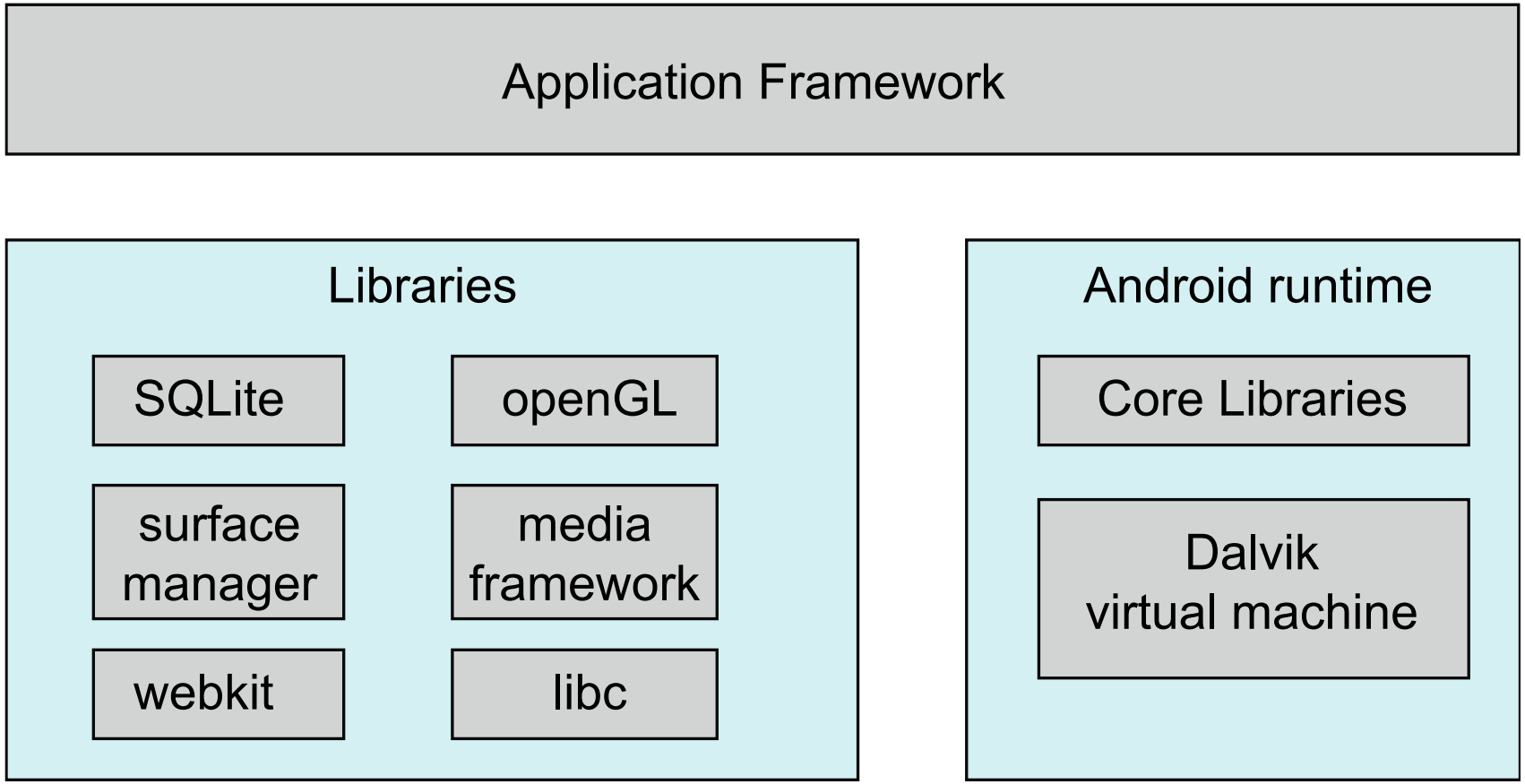
Example in Action

When you **open your Camera app** on an Android phone:

1. The **Application Layer** launches the Camera app.
2. The **Framework Layer** requests access to the camera service.
3. The **Libraries Layer (Media Framework)** processes the image.
4. The **Linux Kernel** communicates with the physical camera hardware.

Everything works together seamlessly.

Android Architecture



Virtual Machines

- **Definition**: OS provides an environment where multiple OS instances run on the same hardware.
- **Real Examples**:
  - **VMware Workstation / ESXi**
  - **VirtualBox**
  - **KVM (Linux Virtualization)**
  - **Microsoft Hyper-V**
- **Daily Life Link**: Like having **separate houses (OS) inside one big building (hardware)**.

Client-Server Model

- **Definition**: OS divided into client processes (request services) and server processes (provide services).
- **Real Examples**:
  - **Minix (client-server based microkernel OS)**
  - **Distributed OS (Amoeba, ChorusOS)**
  - **Modern Cloud Systems** (Google File System, Hadoop)
- **Daily Life Link**: Like **restaurants** → client (customer) requests food, server (kitchen) delivers it.

Comparison table with Real Examples

Structure	Definition	Real Examples
Simple	Mixed, no separation	MS-DOS, Early UNIX
Layered	Divided into layers	THE OS, MULTICS
Monolithic Kernel	All in kernel	Linux, UNIX, MS-DOS
Microkernel	Minimal kernel, rest in user space	Minix, QNX, macOS (partly)
Modular/Hybrid	Monolithic + modular	Modern Linux, Windows NT/10/11, macOS
Virtual Machines	Run multiple OS on one hardware	VMware, VirtualBox, Hyper-V
Client-Server	Client requests, server provides	Minix, Amoeba, ChorusOS

System Calls

System Calls

- A **System Call** is a **bridge (interface)** between:
- **User programs (applications) →** and
- **Operating System (kernel)**
- It allows a **user program** to **request services** from the **OS kernel**, such as reading data, writing files, creating processes, or communicating over the network.

Example
When you use a program to save a file:
1. You click **Save** in MS Word (user-level request).
2. Word can't directly write to the disk (for security reasons).
3. So, it makes a **system call** (e.g., write()).
4. The **OS kernel** performs the actual disk write operation.

System Calls Example

- Compilation Process
  - To compile a program, a process calls the command interpreter or shell, reads command from the terminal
  - Shell will create a new process to compile the program
  - When the process has completed compilation, it executes a system call to terminate itself
- Other process system calls are also available to;
  - Request for more memory
  - Released unused/extra memory
  - Process and memory management

System Calls

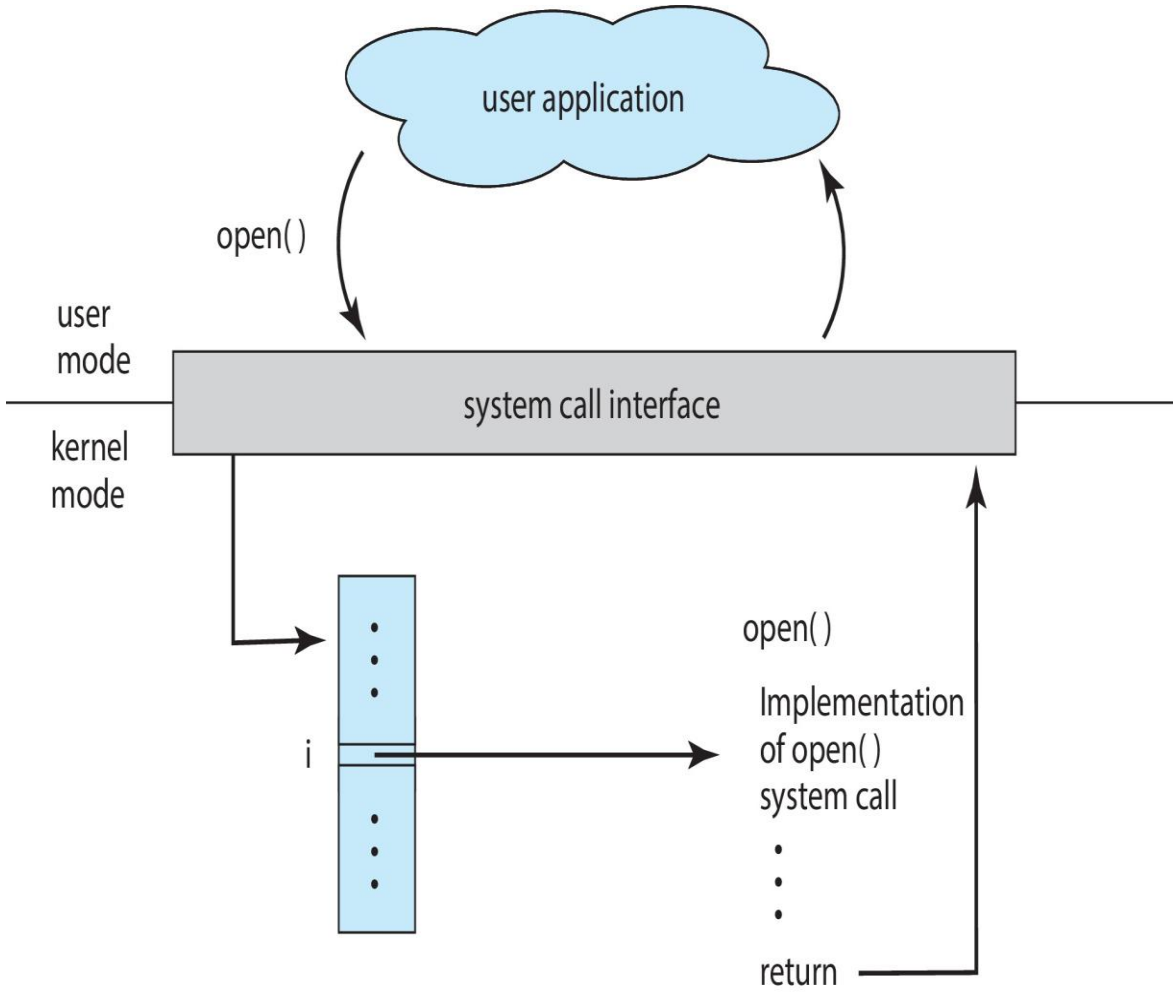
- System calls related to file systems
  - System calls are needed to create, read, move and write a file
    - A file should be opened before read it
    - Similarly, after reading file must be closed which is done by system calls
  - System calls are required to
    - create and remove a directory
    - manipulate file with in different directories
- MS-DOS uses software interrupts similar to system calls

System Call Implementation

- Typically, a number is associated with each system call
  - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

API – System Call – OS Relationship

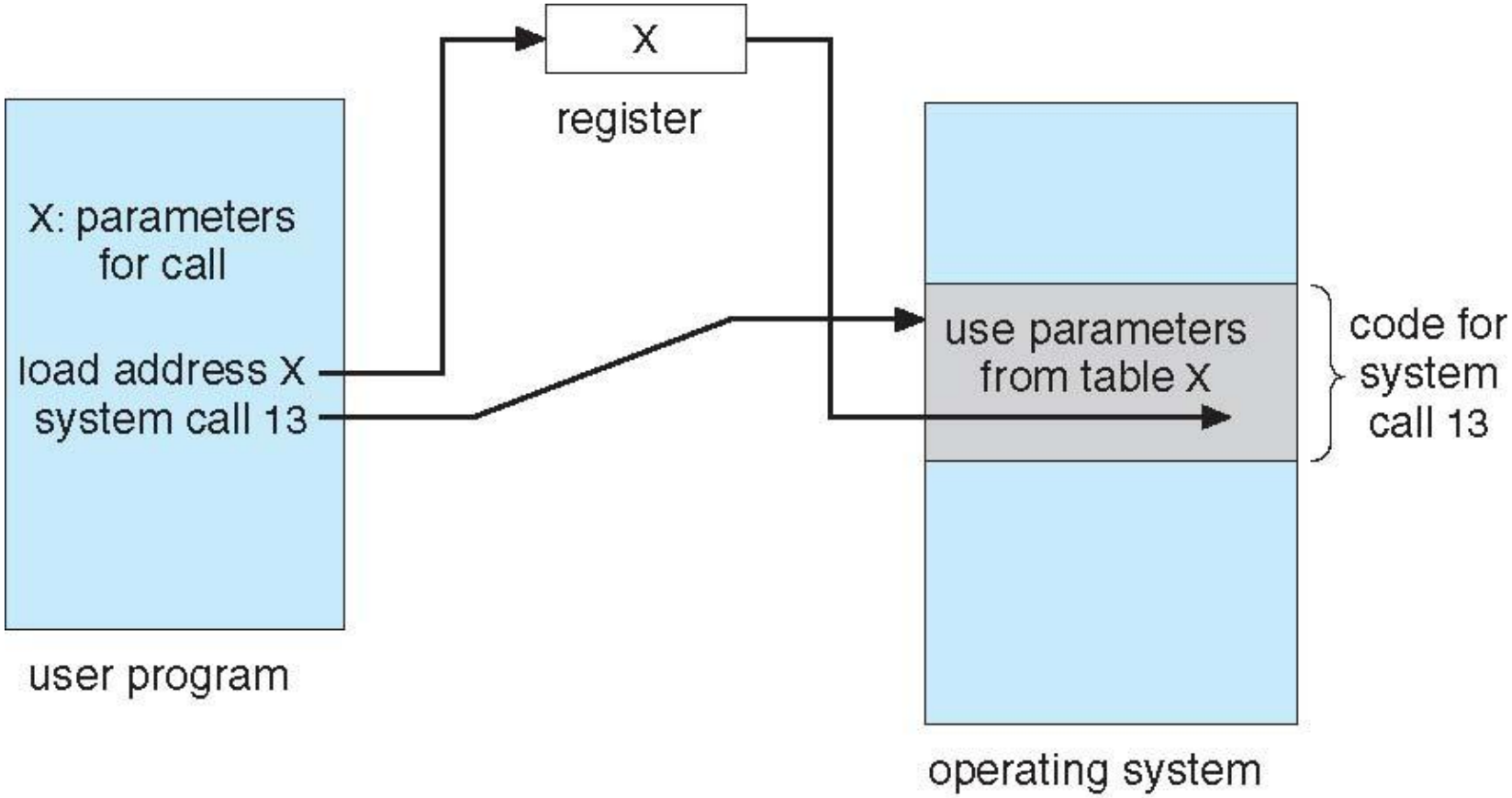
Term	Meaning	Who Uses It
API (Application Programming Interface)	A **set of functions** that a programmer can use to build applications.	**Application developer**
System Call	The **mechanism** that allows an API function to **request services** from the operating system kernel.	**API (indirectly)**
Operating System (OS)	The **core system software** that manages hardware and provides low-level services.	**Kernel-level code**



System Call Processing

- Three general methods are used to pass parameters between a running program and the operating system
  - Pass parameters in registers
  - Store the parameters in a table in memory, and the table address is passed as a parameter in a register
  - Push (store) the parameters onto the stack by the program, and pop off the stack by operating system

System Call Processing



Parameter Passing via Table

Types of System Calls

- Process control
- File management
- Device management
- Information maintenance
- Communications

Types of System Calls

- Process control
  - end, abort
  - load, execute
  - create process, terminate process
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory

  - Dump memory if error
  - Debugger for determining bugs, single step execution
  - Locks for managing access to shared data between processes

Types of System Calls

- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

Types of System Calls

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

Types of System Calls

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Interrupts

Interrupts

- Mechanism by which hardware like I/O devices, memory modules may interrupt the normal processing of the processor
- Interrupts are generated by various agents to notify the OS of the occurrence of some event
  - such as completion of an I/O activity
- When an interrupt occurs, the execution flow of processor is diverted into specific part of OS code which deals with the event – interrupt handler
- Interrupts are used to permit several programs and I/O activities to process independently and asynchronously
  - Hence improve the processing speed

Interrupts

- Advantages
  - It provides a low overheads means of gaining the attention of the CPU
  - Interrupt Vector Table is a location near the bottom of memory contains the address of interrupt services procedures of I/O devices
- As each instruction terminates, the processor may checks for the occurrence of interrupts
  - If an interrupt received, the actual program is temporarily suspended, and the processor is diverted to interrupt handling routine
  - When the interrupt is served, the execution will return to the interrupted program

Interrupts

- Example:
  - External devices like printer are much slower than the processor
  - When processor is transferring data to the printer, the processor pause and remain idle until the printer is reading data
  - I/O modules will send an interrupt request signal to the processor to suspend its current processing and send more data

Interrupt Processing

- Following events occurs in processor hardware and software due to an I/O interrupt
  - The device issue an interrupt signal to processor
  - The processor finishes execution of current instruction, saves states of interrupted process before responding to the interrupt
  - The processor sends an acknowledgment signal to the device that issued the interrupt and device remove its interrupt signal/flag
  - Processor transfer control to the appropriate interrupt handler routine and it starts processing interrupt
  - When the interrupt id served, the state of interrupted process is restored
  - Next process starts execution

Types of Interrupts

Interrupts may be generated by a number of sources, which are following

- I/O:
  - Generated by I/O device controller, to signal normal completion of event or the occurrence of an error or failure condition
- Time:
  - Generated by an internal clock within the processor due to expiration of a process's time quantum or receipt of a signal from another processor on a multiprocessor system
- Hardware error:
  - Generated by hardware faults such as power failure
- Program:
  - Generated in a user program as a result of instruction execution, such as;
    - Arithmetic overflow
    - Divide by zero
    - Attempt to execute an illegal machine instruction
    - Reference outside a user allowed memory space


Any
Question